



AEROLINKERS



A DETAILED DESIGN REPORT

Submitted by

MR. S. NITHILAN, IV-ECE

MS. S. MONISA, II-ECE

MR. T. P. NISHANTH, II-ECE

PRODUCT 360 CONTEST

by

VVDN TECHNOLOGIES PRIVATE LIMITED

K.S. RANGASAMY COLLEGE OF TECHNOLOGY

(Autonomous)

TIRUCHENGODE – 637 215

JANUARY 2025

PRODUCT 360 – ABSTRACT

Team Name	AeroLinkers
College Name	K.S.Rangasamy College of Technology
Participants Name	Mr. S. Nithilan, IV-ECE
	Ms. S. Monisa, II-ECE
	Mr. T. P. Nishanth, II-ECE
Mentor Name	Dr.C.Rajasekaran
Document Version	V1.3
Date	20-01-2026

1. ARCHITECTURE (*SECTION 1*)

1.1 ARCHITECTURE OVERVIEW AND JUSTIFICATION

The system architecture is designed as a **low-power, event-driven wearable platform** optimized for intermittent connectivity, continuous sensing, and short duration user interactions. The architecture separates **always on ultra-low power functions** from **high energy, event-based operations** to achieve predictable battery life and reduced idle power consumption.

A centralized power management subsystem controls multiple regulated domains, enabling selective power gating of high current peripherals.

1.2 HARDWARE ARCHITECTURE

The hardware architecture consists of:

- A single ultra-low-power MCU responsible for system control
- A companion Wi-Fi IC activated only during connectivity events
- A Power Management Integrated Circuit with battery charging and regulation
- Dedicated subsystems for display, touch, audio, haptics, and motion sensing

Each hardware block is supplied through controlled power rails and communicates with the MCU using standard digital interfaces.

(Detailed design is covered in Section 2)

1.3 SOFTWARE ARCHITECTURE

The firmware architecture employs a layered RTOS-based design with clear separation between boot-time execution, system management services, hardware drivers, and application-level logic. System behaviour is governed by event notifications rather than continuous polling, minimizing CPU wake ups. Peripheral power states are dynamically managed within the scheduler context to reduce idle current and avoid unnecessary resource activation.

The system software is designed to interoperate with the **ATAB Server** for selected cloud-assisted functions such as configuration synchronization, device registration, and event data exchange. ATAB Server APIs are consumed strictly at the **application layer**, ensuring that core device functionality remains operational in offline conditions.

The integration follows a **client-initiated, event-based communication model**, where the device establishes connectivity only when required. No continuous background synchronization is implemented. This design minimizes power consumption and aligns with the wearable's event-driven architecture.

ATAB APIs are accessed over a secure IP channel using standard request–response semantics. The API layer is isolated from hardware drivers and RTOS services to prevent tight coupling and to allow future server-side evolution without impacting low-level firmware.

(Detailed design is covered in Section 3)

1.4 MECHANICAL ARCHITECTURE

The mechanical architecture is designed for a **compact wrist-worn form factor**, integrating the PCB, battery, display, and haptic components within a sealed enclosure suitable for daily wearable use.

(Detailed design is covered in Section 4)

2. HARDWARE DETAILED DESIGN (*SECTION 2*)

2.1 COMPONENT SELECTION MATRIX

This section documents all **key hardware components** selected for the system along with justification.

Table 2.1.1 – MCU Comparison and Selection

Parameter	STM32U375RG (Selected)	nRF5340	CC3551E	STM32U5
MCU type	Ultra-low-power MCU	Radio-centric SoC	Wi-Fi MCU SoC	Feature MCU
Integrated radio	None	BLE	Wi-Fi	None
Active current @ 64 MHz	~32 μ A/MHz	~55 μ A/MHz	~48 μ A/MHz	~40 μ A/MHz
Deep sleep current	~1.6 μ A	~2.2 μ A	~2.8 μ A	~1.9 μ A
Wake-up latency	<10 ms	~15 ms	~18 ms	~12 ms
Flash	Up to 1 MB	512 KB–1 MB	1 MB	512 KB–2 MB
SRAM	Up to 256 KB	256 KB	256 KB	128–786 KB
Lead time	6–8 weeks	10–12 weeks	12–14 weeks	8–10 weeks

MCU Justification



STM32U375RG MCU

STM32U375RG was selected because it delivers the **lowest energy per task**, the **lowest deep sleep current**, and **no RF related idle leakage**, while providing sufficient on-chip memory and certified hardware security. MCUs with integrated radios were rejected due to higher idle power, RF coexistence risk, and increased firmware complexity unsuitable for an event driven wearable.

Table 2.1.2 – Wi-Fi Solution Comparison

Parameter	nRF7002 (Selected)	CC3351	ESP32-C6
Wi-Fi architecture	Companion IC	Companion IC	SoC
Wi-Fi standard	Wi-Fi 6 (2.4/5 GHz)	Wi-Fi 6	Wi-Fi 6
Sleep current	<5 μ A	~8 μ A	~20 μ A
TX peak current	262 mA	400 mA	480 mA
Energy per TX event	Lowest	22%	45%
Interface	SPI	SPI	Internal
10k unit cost	~\$4.8–5.2	~\$5.6–6.0	~\$2.8–3.2

Wi-Fi Justification**nRF7002 Wi-Fi 6 Companion IC**

nRF7002 provides the **lowest Wi-Fi energy per event** among companion class devices while allowing **complete power gating when idle**. SoC based Wi-Fi solutions were rejected due to excessive idle current and increased firmware and certification complexity incompatible with smartwatch class power budgets.

Table 2.1.3 – PMIC Comparison

Parameter	nPM1300 (Selected)	MAX20353	nPM1304
Integrated charger	Yes	Yes	Yes
Buck regulators	1 programmable	2 (limited)	1
Max buck voltage	3.3 V	3.85 V	3.3 V
Wearable optimization	Yes	Medium	Medium
System complexity	Low	Medium	Low
Rail flexibility	Limited	Medium	Limited
Lead time	6–8 weeks	8–10 weeks	6–8 weeks

PMIC Justification



NPM1300 PMIC

nPM1300 was selected for its **low complexity wearable class integration**, combining charging, regulation, and fuel gauging. Although it offers limited rail flexibility, system power domains are consolidated to avoid unnecessary regulators, reducing BOM and integration risk.

Table 2.1.4 – IMU Comparison

Parameter	LSM6DSOX (Selected)	BMI270	ICM-42670
Accel LP current	~12 μ A	~14 μ A	~15 μ A
Normal accel current	~550 μ A	~685 μ A	~720 μ A
Embedded intelligence	MLC + FSM	FSM only	None
MCU wake reduction	Highest	Medium	Low
Energy per wake event	Lowest	+25%	+30%
Lead time	6–8 weeks	10–12 weeks	12–14 weeks

IMU Justification



LSM6DSOX 6-Axis IMU

LSM6DSOX was selected due to its **embedded motion intelligence**, allowing motion decisions without MCU wake ups, resulting in the **lowest always-on power consumption** and reduced firmware complexity.

Table 2.1.5 – Audio Codec Comparison

Parameter	SGTL5000 (Selected)	MAX98090	TLV320AIC3204
Idle	~6 mA	~9 mA	~7 mA
Voice playback current	20–25 mA current	~30 mA	~28 mA

Audio Justification

SGTL5000 Stereo Codec Chip

SGTL5000 was selected for **voice-prompt-centric usage**, offering the **lowest energy per alert** and minimal analog and firmware complexity. Music-grade codecs were rejected as over-engineered for Gen-1 wearable requirements.

Table 2.1.6 – Haptic Driver Comparison

Parameter	DRV2603 (Selected)	DRV2625	AW8697
Motor type	LRA	LRA	LRA + ERM
Control	Closed-loop	Closed-loop	Semi-open
Idle current	~0.5 mA	~1.0 mA	~1.5 mA

Haptic Justification

DRV2603 was selected for its **voltage independent vibration consistency**, lowest idle current, and minimal firmware involvement, making it ideal for accessibility alerts in a wearable device.

Display and Touch Summary

- **Display:** 1.69" IPS TFT (280×240)
- **Driver:** ST7789V2

- **Touch:** CST816D (single-touch)

This combination provides:

1. Predictable power consumption
2. Mature driver ecosystem
3. Lowest BOM and validation risk for Gen-1

This section describes all digital and analog interfaces.

2.2 INTERFACES

The system interfaces are designed to support an event-driven, ultra-low-power wearable architecture while maintaining electrical simplicity, signal integrity, and firmware robustness. All interfaces operate at a nominal logic voltage of 2.7 V, eliminating the need for level translation and reducing static and dynamic power consumption.

High-speed interfaces are limited to short-duration, on-demand activity to minimize EMI and energy loss, while low-speed control interfaces remain idle for the majority of system lifetime. Interface selection prioritizes mature protocols, wide ecosystem support, and deterministic timing behavior, ensuring reliable operation in a compact smartwatch-class form factor.

All interfaces are electrically isolated through power-domain control, ensuring that inactive subsystems do not introduce leakage paths or unintended loading. This interface strategy provides a balanced trade-off between **power efficiency, signal integrity, and system scalability**,

Table 2.2 – Interface Description

Interface	Source	Target	Voltage	Frequency / Data Rate	Purpose
SPI-1	MCU	IMU	1.8–3.3 V	Up to 10 MHz	Motion data
SPI-2	MCU	Display	1.8–3.3 V	20–40 MHz	UI rendering
SPI-QSPI	MCU	Wi-Fi IC	1.8–3.3 V	upto100 MHz	Network data
I²C-1	MCU	PMIC	1.8 V	400 kHz	Power control
I²C-2	MCU	Touch	3.3 V	400 kHz	Touch input

I²S / SAI	MCU	Audio codec	1.8–3.3 V	Audio clocks	Voice output
GPIO	MCU	Haptic / Button	3.3 V	—	User feedback

2.3 DC & AC ANALYSIS

2.3.1 Power Rails – DC Analysis

Power Rail	Source	Nominal Voltage	Min–Max	Max Load Current	Loads Connected
VBAT	Li-Po Battery	3.7 V	3.0–4.2 V	800 mA	PMIC
VSYS	nPM1300	Unregulated	VBAT – PMIC drop	1 A	All rails
VOUT-1	nPM1300 Buck	2.7 V	2.62–2.78 V	200 mA	Display + LED driver
VOUT-2	nPM1300 Buck	2.7 V	2.62–2.78 V	200 mA	Wi-Fi nRF7002
VDD_LDO	TPS76201	2.7 V	±2%	100 mA	MCU, IMU, Touch, Audio
LED Rail	CC Driver	Current mode	—	40 mA	Backlight

2.3.2 MCU GPIO – DC Logic Levels @ 2.7 V

Parameter	Value	Calculation / Basis
VDD	2.7 V	System rail
VIH (min)	$\geq 1.9 \text{ V}$	$0.7 \times \text{VDD}$
VIL (max)	$\leq 0.8 \text{ V}$	$0.3 \times \text{VDD}$
VOH (min)	$\geq 2.5 \text{ V}$	$\text{VDD} - 0.2 \text{ V}$
VOL (max)	$\leq 0.2 \text{ V}$	CMOS spec
Max GPIO current	8 mA	Configured per pin
Internal pull-up	30–50 k Ω	MCU default

All peripherals selected support **2.7 V logic directly**, no level shifting required.

2.3.3 SPI Interface – DC & AC @ 2.7 V

(Display + Wi-Fi)

Parameter	SPI-Display	SPI-Wi-Fi
Logic voltage	2.7 V	2.7 V
Clock frequency	40 MHz	32 MHz
VIH requirement	$\geq 1.9 \text{ V}$	$\geq 1.9 \text{ V}$
Rise/Fall time	<4 ns	<4 ns
Trace impedance	$\sim 50 \Omega$	$\sim 50 \Omega$
Series termination	22–33 Ω	33 Ω
AC load	Moderate	High (TX bursts)

AC justification at 2.7 V

- Slightly slower edges vs 3.3 V → lower EMI
- SPI margin remains >25% at 40 MHz with controlled routing

2.3.4 I²C Interface – DC & AC @ 2.7 V

(IMU, Touch, Audio control)

Parameter	Value
Bus voltage	2.7 V
Speed	Fast-mode (400 kHz)
Pull-up resistance	4.7 k Ω
Pull-up current	≈ 0.57 mA (2.7 V / 4.7 k Ω)
Max sink capability	>3 mA (all devices)
Rise time (tr)	<300 ns
Fall time (tf)	<300 ns
Bus capacitance	<200 pF

Pull-up current is **intentionally kept <1 mA** to reduce static loss.

2.3.5 I²S Interface – AC Analysis @ 2.7 V

(MCU $\leftrightarrow$ SGTL5000)

Parameter	Value
Logic voltage	2.7 V
BCLK	1–3 MHz
LRCLK	16–48 kHz
Data width	16 / 24-bit
Clock master	MCU
Edge timing	Data valid on rising edge
Jitter tolerance	<100 ps

AC note: Short trace length and reduced swing at 2.7 V significantly lower digital audio EMI.

2.3.6 Interrupt Lines – DC & AC @ 2.7 V

Source	Type	Voltage	Trigger	Pull-up	Priority
User Button	GPIO	2.7 V	Falling edge	Internal	Level-1
Touch INT	Open-drain	2.7 V	Falling edge	10 kΩ	Level-2
IMU INT	Push-pull	2.7 V	Rising edge	None	Level-3
Wi-Fi IRQ	Push-pull	2.7 V	Level	None	Level-4

2.3.7 Analog & Mixed-Signal Stability @ 2.7 V

Subsystem	Sensitivity	Supply	PSRR Needed	Justification
IMU	High	LDO @ 2.7 V	>60 dB	Motion noise immunity
Audio Codec	Very High	LDO @ 2.7 V	>65 dB	Audible noise prevention
Touch	Medium	LDO @ 2.7 V	>50 dB	Gesture reliability
Display Logic	Low	Buck @ 2.7 V	>40 dB	Digital tolerant

2.3.8 EMI & Signal Integrity Summary (2.7 V System)

Aspect	Implementation
Reduced voltage swing	Naturally lower EMI
High-speed SPI	Series damping + short return loops
Audio isolation	Separate LDO + ground island
Wi-Fi bursts	Dedicated buck rail
ESD	TVS on external interfaces

2.3.9 DC & AC Compliance Statement

All digital and analog interfaces operate at 2.7 V nominal, fully within device-specified limits. Signal integrity margins are preserved despite lower swing, while EMI and static power are reduced. This voltage strategy is optimal for an event-driven, Gen-1 wearable and consistent with the locked power architecture.

2.4 POWER SEQUENCING

Power-Up Sequence

- Battery or charger applied to PMIC
- PMIC generates VSYS
- Primary buck enabled (MCU + display)
- MCU boots and configures PMIC
- Secondary rails enabled for audio, IMU, haptic
- Wi-Fi rail enabled only when required

Power-Down Sequence

- Reverse order, Wi-Fi disabled first

2.5 POWER ARCHITECTURE

- Single-cell Li-Po battery
- PMIC buck generates main system rail
- Secondary LDO provides low-noise rail
- Separate power gating for Wi-Fi and backlight
- Battery voltage and current monitored by PMIC

2.6 POWER CONSUMPTION & DISSIPATION CALCULATIONS

Typical Current Consumption

Operation (Approx. time)	PMIC and LDO	MCU	Wi-Fi	DISPLAY & Back Light (50% PWM)	IMU	AUDIO	HAPTIC (50% PWM)	TOTAL
Booting (5 Sec)	0.823	3	180	50	0.55	5	30	269.37
Data Communication (1 Sec)	0.823	3	60	0	0.55	5	30	99.37
Rest Position	0.823	3	60	0	0.55	0	0	94.37
Wake Position	0.823	3	60	50	0.55	0	0	114.37
Docking	0.823	3	60	0	0.55	0	0	64.37
Vibration (1 Sec)	0.823	3	60	50	0.55	0	30	144.37

(All values in mA)

Estimated runtime: **18–24 hours** typical mixed usage.

2.7 RESET SCHEME

- Power-on reset via PMIC
- Software reset controlled by MCU
- Watchdog reset for fault recovery
- User action button triggers application-level reset

2.8 GPIO MAPPING

Table 2.8 – GPIO Mapping

MCU Pin	Signal Name	Direction	Connected Subsystem	Interface	Description
PB6	I2C1_SCL	Bi-dir	PMIC (nPM1300)	I2C-1	PMIC control, charger & fuel- gauge
PB3	I2C1_SDA	Bi-dir	PMIC (nPM1300)	I2C-1	PMIC control, charger & fuel- gauge
PB7	PMIC_SHPHLD	Output	PMIC	GPIO	Ship-hold / system power latch

PC9	USER_BTN	Input	User Button	GPIO / EXTI	User action interrupt (internal pull- up)
PB0	SPI1_CS	Output	IMU (LSM6DSOX)	SPI-1	IMU chip select
PA7	SPI1_SCK	Output	IMU (LSM6DSOX)	SPI-1	IMU SPI clock
PA6	SPI1_MISO	Input	IMU (LSM6DSOX)	SPI-1	IMU data to MCU
PB1	IMU_INT1	Input	IMU (LSM6DSOX)	GPIO / EXTI	Motion / wake interrupt
PB2	IMU_INT2	Input	IMU (LSM6DSOX)	GPIO / EXTI	Secondary motion interrupt
PC7	HAPTIC_EN	Output	Haptic Driver (DRV2603)	GPIO	Enables haptic driver
PC8	HAPTIC_PWM	Output	Haptic Driver (DRV2603)	PWM	Vibration amplitude control
PB10	WIFI_CLK	Output	Wi-Fi (nRF7002)	QSPI-1	Clock
PA4	WIFI_SS	Output	Wi-Fi (nRF7002)	QSPI-1	Slave select
PC10	WIFI_D0_MOSI	Output	Wi-Fi (nRF7002)	QSPI-1	Data 0 / MOSI
PC12	WIFI_D1_MISO	Input	Wi-Fi (nRF7002)	QSPI-1	Data 1 / MISO
PD2	WIFI_D2	Bi-dir	Wi-Fi (nRF7002)	QSPI-1	Data line 2
PC6	WIFI_D3	Bi-dir	Wi-Fi (nRF7002)	QSPI-1	Data line 3
PC5	WIFI_IRQ	Input	Wi-Fi (nRF7002)	GPIO / EXTI	Wi-Fi interrupt
PC0	I2C3_SCL	Bi-dir	Audio Codec (SGTL5000)	I2C-3	Codec configuration
PC1	I2C3_SDA	Bi-dir	Audio Codec (SGTL5000)	I2C-3	Codec configuration
PA7	I2S_DOUT	Input	Audio Codec (SGTL5000)	SAI-1 / I2S	Mic data to MCU
PB5	I2S_DIN	Output	Audio Codec (SGTL5000)	SAI-1 / I2S	Audio data to codec
PA5	I2S_SCLK	Output	Audio Codec (SGTL5000)	SAI-1 / I2S	Serial clock
PA4	I2S_LRCLK	Output	Audio Codec (SGTL5000)	SAI-1 / I2S	Frame sync
PC3	SPI2_DIO	Output	Display (ST7789V2)	SPI-2	Display data

PA9	SPI2_SCK	Output	Display (ST7789V2)	SPI-2	Display clock
PB12	SPI2_CS	Output	Display (ST7789V2)	SPI-2	Display chip select
PB13	TOUCH_I2C_SDA	Bi-dir	Touch (CST816D)	I2C-2	Touch data
PB14	TOUCH_I2C_SCL	Bi-dir	Touch (CST816D)	I2C-2	Touch clock
PC2	DISP_RST	Output	Display	GPIO	Display reset
PA0	LED_PWM_BL	Output	Backlight Driver	PWM	LED backlight current control

2.9 INTERRUPT MAPPING

Priority Order

- User action button
- Touch interrupt
- IMU motion interrupt
- Wi-Fi interrupt

Interrupts are edge-triggered and handled by RTOS ISR callbacks.

3. SOFTWARE DETAILED DESIGN (*SECTION 3*)

3.1 CPU SELECTION MATRIX

(Condensed comparison table retained here, detailed analysis already validated)

STM32U375RG selected due to:

- Lowest $\mu\text{A}/\text{MHz}$
- No RF leakage
- Adequate memory
- Certified security features

3.2 MEMORY REQUIREMENTS ANALYSIS & ESTIMATION

This section analyzes the non-volatile and volatile memory requirements of the system firmware and validates that the selected MCU provides sufficient margin for current functionality and future scalability. Memory estimation is based on **actual functional blocks, RTOS overhead, and worst-case concurrent execution**, rather than optimistic averages.

The system is designed to avoid external memory, reducing power consumption, BOM cost, and integration risk.

3.2.1 MCU Memory Availability (Reference)

Memory Type	Available on MCU
Internal Flash	512 KB
Internal SRAM	256 KB
External Memory	Not used

3.2.2 Flash Memory Requirement Analysis

Flash memory is used for bootloader, RTOS kernel, device drivers, application code, configuration tables, and OTA support.

Flash Usage Category	Estimated Size (KB)	Basis
Secure Bootloader	32 KB	Image validation, fallback logic
Zephyr RTOS Kernel	60 KB	Scheduler, IPC, power mgmt
HAL & BSP	45 KB	MCU peripherals
Device Drivers	85 KB	Display, touch, IMU, audio, haptic, PMIC
Wi-Fi Stack & Driver	70 KB	SPI-based companion control
Application Logic	90 KB	UI, alerts, navigation logic
Configuration & Tables	20 KB	Fonts, UI strings, language
Diagnostics & CLI	25 KB	SSH-based CLI
Subtotal	427 KB	
Flash Safety Margin (~15%)	65 KB	Future updates
Total Required	≈ 492 KB	

Conclusion (Flash):

The 512 KB internal flash provides sufficient capacity with controlled margin. Dual-image OTA is **possible but constrained**; Gen-1 implementation uses **single-image + rollback protection** to preserve margin.

3.2.3 RAM Requirement Analysis

RAM usage includes RTOS kernel data, stacks for concurrent threads, heap allocations, DMA buffers, and frame buffers.

a) RTOS & System Overhead

Component	RAM Usage (KB)
Zephyr kernel objects	12 KB
IPC queues & events	8 KB
Timer & scheduler data	6 KB
Subtotal	26 KB

b) Thread Stack Allocation (Worst-Case)

Thread / Task	Stack Size (KB)
Main application thread	16 KB
UI / Display task	20 KB
Wi-Fi control task	18 KB
Audio playback task	16 KB
Sensor / IMU task	12 KB
CLI / Diagnostics task	14 KB
Idle / system task	6 KB
Subtotal	102 KB

c) Dynamic Buffers & DMA

Buffer Type	Size (KB)
Display line/frame buffers	32 KB
Audio DMA buffers	16 KB
Wi-Fi SPI buffers	24 KB
Sensor data buffers	8 KB
Subtotal	80 KB

d) Heap & Safety Margin

Category	Size (KB)
Heap allocation	20 KB
RAM safety margin (~10%)	20 KB

3.2.4 Total RAM Estimation Summary

Category	RAM (KB)
RTOS & kernel	26
Thread stacks	102
Buffers & DMA	80
Heap	20

Safety margin	20
Total Required	248 KB

Conclusion (RAM):

The estimated RAM usage fits within the 256 KB internal SRAM with tight but acceptable margin. Stack sizes are intentionally conservative to avoid runtime overflow. No external RAM is required.

3.2.5 Memory Optimization Strategy

To maintain margin and reliability:

- Display uses **line buffering**, not full-frame buffering
- Wi-Fi buffers allocated only during active sessions
- Audio buffers released immediately after playback
- Static allocation preferred over heap where possible
- Stack usage monitored using RTOS watermarking

3.2.6 Memory Sufficiency Conclusion

The selected MCU memory configuration is **right-sized** for the Gen-1 wearable:

- No excess memory causing leakage
- No external memory integration risk
- Sufficient margin for firmware refinement and feature tuning

This memory strategy aligns with the product goal of **low power, low complexity, and high reliability**.

3.3 IDE, OS & DEVICE DRIVERS

- IDE: STM32CubeIDE
- RTOS: Zephyr
- Device drivers: Native Zephyr drivers
- CLI access via SSH using WolfSSH

3.4 BOOT FLOW

1. Power-on reset
2. Secure boot validation
3. RTOS initialization
4. Peripheral bring-up
5. Application start

3.5 KEY PERIPHERAL INITIALIZATION FLOW

1. Clock configuration
2. GPIO setup
3. PMIC configuration
4. IMU and touch init
5. Display init
6. Audio and haptic init

3.6 LIST OF APPLICATIONS

- Device information
- Language selection
- Display settings
- Audio & haptic alerts
- Flight / navigation alerts
- Power and diagnostics

3.7 ATAB SERVER API INTEGRATION

3.7.1 Purpose of ATAB Server Integration

ATAB Server APIs are used to support **non-time-critical, non-safety-critical functions** such as:

- Device identification and provisioning
- Retrieval of user- or session-specific data
- Upload of device-generated events (status, alerts)
- Configuration updates

All core wearable functions (navigation cues, alerts, haptics, display, audio) **do not depend** on continuous ATAB server availability.

3.7.2 Communication Model

Aspect	Description
Communication Type	Client-initiated
Transport	IP-based (over Wi-Fi)
Session Type	Short-lived
Connectivity Mode	Event-triggered
Retry Strategy	Limited retries with timeout
Offline Behavior	Graceful degradation

The device initiates communication with ATAB Server only when:

- User action explicitly requires data
- A scheduled sync window is reached
- Configuration refresh is requested

3.7.3 ATAB API Usage Context

API Category	Device Usage Context
Device Registration	First boot / factory reset
Device Status API	On-demand health reporting
Configuration API	Language, preferences, alerts
Event Upload API	Post-event synchronization
Authentication APIs	Session establishment

No ATAB API is invoked from ISR context or real-time tasks.

3.7.4 Software Layer Placement

Layer	Responsibility
Application Layer	API request formation, response parsing
Middleware	JSON parsing, message validation
Network Stack	TCP/IP, TLS
RTOS / HAL	Scheduling, memory, drivers

ATAB APIs are never accessed directly by device drivers.

3.7.5 3.X.5 Security Considerations

- Secure session establishment prior to API access
- Credentials stored in MCU secure storage
- API access restricted to authenticated sessions
- No sensitive data stored persistently unless required

Security implementation aligns with MCU hardware cryptographic capabilities.

3.7.6 Failure Handling Strategy

Failure Case	Device Behavior
Server unreachable	Local operation continues
API timeout	Retry with backoff

Authentication failure	User notification (if required)
Invalid response	Response discarded, logged

Failure of ATAB APIs **does not block user-facing functions**.

3.8 COMMAND LINE INTERFACE

- Access via Wi-Fi
- SSH-based secure CLI
- Used for diagnostics, logs, and configuration

3.8.1 ATAB Diagnostic Commands

ATAB integration is supported by CLI-level diagnostics for development and testing purposes.

Command	Description
<code>atab_status</code>	Check server connectivity
<code>atab_auth</code>	Validate authentication
<code>atab_sync</code>	Trigger manual sync
<code>atab_log</code>	Display last API transaction

These commands are **disabled or restricted** in production builds.

4. MECHANICAL DETAILED DESIGN (*SECTION 4*)

4.1 Exploded View of Concept Design

- Front glass + display
- Touch sensor
- Main PCB
- Battery

- Haptic motor
- Enclosure and strap

Assembly sequence ensures minimal rework and safe battery handling.

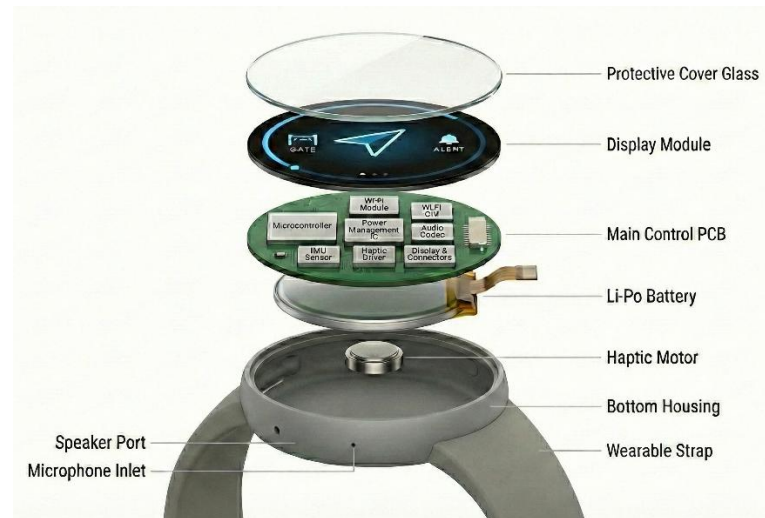


Fig 4.1 Exploded view

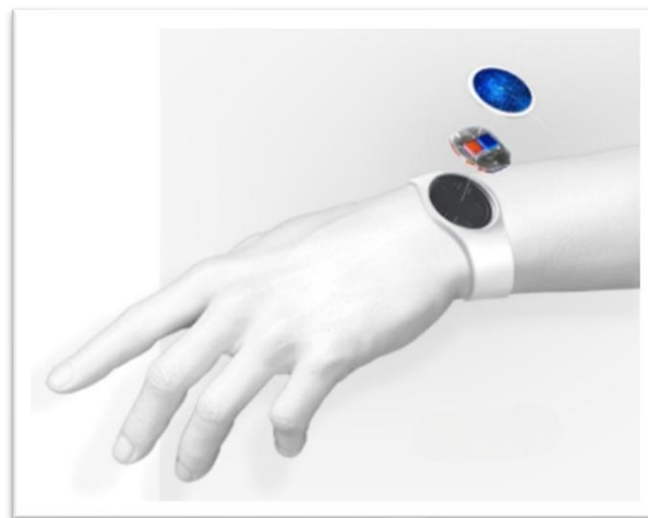


Fig 4.2 Exploded view in Hand Wrist

GitHub Repository

